

Computer Programming (NCSV101) [Winter 2025-26]

Extra Slides# 1

Conversion Between Number System

SAURABH SRIVASTAVA

ASSISTANT PROFESSOR

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

IIT (ISM) DHANBAD

Important Note

You will see Lecture Slides like this one once in a while, starting with prefix “E”

These slides may not be directly related to your Syllabus

- But they are important for your overall understanding

It is strongly suggested that you go through these slides

Why Number System Conversion Is Needed?

Humans are comfortable with decimal (base 10) numbers

Computers internally work with binary (base 2) numbers

So, whenever we

- Enter a number using a keyboard, and see a result on the screen ...
- ... there must be some “conversion” going on in the background somewhere

The key idea you need to understand – *A number’s value remains the same*

- Number System conversion only *changes its representation*

For example

- 13 – in Base 10 – can also be written as $(13)_{10}$, is equivalent to
- 1101 – in Base 2 – can also be written as $(1101)_2$, is equivalent to
- 15 – in Base 8 – can also be written as $(15)_8$, is equivalent to
- D – in Base 16 – can also be written as $(D)_{16}$ (Note that the digit “10” is “A”, “11” is “B”, and so on)

Converting from Binary to Decimal

The Binary system has only 2 digits – 0 and 1

- In fact, any number system with base n has precisely n digits

Each digit in a binary representation essentially represents a power of 2

- For example, the number $(01101)_2$ is essentially

2^4	2^3	2^2	2^1	2^0
0	1	1	0	1

To find the decimal representation of a binary number, we need to do some basic arithmetic

- Each digit needs to be multiplied with the power of 2 that it represents
- For example, $(01101)_2 \rightarrow (2^4 * 0 + 2^3 * 1 + 2^2 * 1 + 2^1 * 0 + 2^0 * 1)_{10} \rightarrow (0 + 8 + 4 + 0 + 1)_{10} \rightarrow (13)_{10}$
- Since there are only two possibilities of a digit here – 0 or 1 – we can safely say the following:
The decimal representation of a number can be found from its binary representation by a summation of the powers of 2 corresponding to the positions in the binary representation having the digit 1

Converting from *any base* to Decimal

The process to convert a representation of a number in any other base to Decimal is similar

- We need to represent the number as a summation of correct product terms
- Each product term multiplies the digit with the corresponding power of the base

Example: Octal (base 8) to decimal

$$(107)_8 \rightarrow (8^2 * 1 + 8^1 * 0 + 8^0 * 7)_{10} \rightarrow (64 + 0 + 7)_{10} \rightarrow (71)_{10}$$

Example: Hexadecimal (base 16) to decimal [Note that F represents the digit 15]

$$(1F6)_{16} \rightarrow (16^2 * 1 + 16^1 * 15 + 16^0 * 6)_{10} \rightarrow (256 + 240 + 6)_{10} \rightarrow (502)_{10}$$

Note that unlike Binary Representation, where summing the relevant powers directly was fine ...

... in general, you will have to multiply the corresponding power with the corresponding digit as well

Converting from Decimal to Binary – Idea

Intuitively, the process of finding the Binary Representation of a number, say n , is summarised as

- Find the largest power of 2 which is smaller or equal to the number, let this be 2^k ; thus $2^k \leq n < 2^{k+1}$
- This means that we will need at least k digits to represent this number in Binary Representation
- It also means that the digit corresponding to the k^{th} position is 1 leading to the product term $2^k * 1$
- We should now find other $k-1$ digits; to do so, subtract 2^k from the number, and find its Binary Representation
- It is *guaranteed* that this representation will take at most $k-1$ digits, which are the other $k-1$ digits we want

What we just did is finding solution to a larger problem in terms of a smaller problem

- For example, "binary representation of n " = "1" followed by "binary representation of $n - 2^k$ in $k-1$ digits"
- Note the "in $k-1$ digits" part here, since this number may be significantly smaller than 2^{k-1} ...
- ... which in turn, means that we may have to add 0s to its left to make it $k-1$ digits long
- This type of problem-solving technique is called *Recursion*; we will get back to it in detail later

Example: $(13)_{10} \rightarrow (1 \text{ followed by } 13 - 2^3 \text{ in 3 digits})_2 \rightarrow (11 \text{ followed by } 5 - 2^2 \text{ in 2 digits})_2 \rightarrow (1101)_2$

Converting from Decimal to Binary – Steps

Now that you know the idea, let us look at the steps to do the same by the **Repeated Division**

- Divide the number by 2
- Write down the remainder (0 or 1)
- Divide the quotient again by 2
- Repeat until the quotient becomes 0
- Read the remainders from bottom to top
- Example: $(13)_{10}$

Division	Quotient	Remainder
$13 \div 2$	6	1
$6 \div 2$	3	0
$3 \div 2$	1	1
$1 \div 2$	0	1

Reading the remainders: $(1101)_2$
(from bottom to top)

Converting from Decimal to *any base*

The ideas and the method is the same – just use the base instead of 2 for the operations

- Example: $(71)_{10}$ to Octal

Division	Quotient	Remainder
$71 \div 8$	8	7
$8 \div 8$	1	0
$1 \div 8$	0	1

Reading the remainders: $(107)_8$
(from bottom to top)

- Example: $(502)_{10}$ to Hexadecimal

Division	Quotient	Remainder
$502 \div 16$	31	6
$31 \div 16$	1	$15 \equiv F$
$1 \div 16$	0	1

Reading the remainders: $(1F6)_{16}$
(from bottom to top)